

Laboratorium
Robotika dan Sistem Cerdas
Departemen Teknik Komputer
Institut Teknologi Sepuluh Nopember

Laporan Akhir
Praktikum Sistem Mikroprosesor dan Sistem Tertanam

Pengantar Mikroprosesor

Kun Alimul Ibad - 5024241026

2026

1 Langkah-Langkah Percobaan

1.1 Pengenalan STM32 Blackpill (Blink)

1. Ganti file platform.ini pada project menjadi seperti berikut.

```
[env:blackpill_f401cc]
platform = ststm32
board = blackpill_f401cc
framework = arduino
upload_protocol = stlink
debug_tool = stlink
```

2. Tunggu hingga konfigurasi selesai.

3. Hubungkan ST Link ke STM32. Wiring dapat merujuk pada tabel berikut.

Pin ST-Link	Pin Blackpill	Fungsi
SWDIO	SWDIO	Jalur data/debug
SWCLK	SWCLK	Jalur clock/debug
GND	GND	Referensi ground bersama
3.3V	3V3	Referensi/tegangan target board

4. Hubungkan ST Link ke laptop.

5. Tulis kode Blink di bawah pada src/main.cpp, lalu klik Build dan Upload

```
#include <Arduino.h>

#define LED_PIN PC13

void setup()
{
  pinMode(LED_PIN, OUTPUT);
}

void loop()
{
  digitalWrite(LED_PIN, LOW);  // LED onboard aktif LOW -> LOW berarti menyala
  delay(500);
  digitalWrite(LED_PIN, HIGH); // HIGH berarti mati
  delay(500);
}
```

1.2 Pengenalan ESP32 dengan Framework ESP-IDF (Blink)

1. Ganti file platform.ini pada project menjadi seperti berikut.

```
[env:esp32dev]
platform = espressif32
board = esp32dev
framework = espidf
```

2. Colokkan ESP32 ke komputer lewat USB, lalu pastikan komputer mengenali port-nya

3. Hapus file src/main.cpp, buat file baru src/main.c, tulis kode Blink di bawah.

```
#include "freertos/FreeRTOS.h"
#include "freertos/task.h"
#include "driver/gpio.h"

#define LED_PIN GPIO_NUM_2

void app_main(void)
{
  gpio_reset_pin(LED_PIN);
  gpio_set_direction(LED_PIN, GPIO_MODE_OUTPUT);

  while (1)
  {
    gpio_set_level(LED_PIN, 1);
    vTaskDelay(pdMS_TO_TICKS(500));
    gpio_set_level(LED_PIN, 0);
    vTaskDelay(pdMS_TO_TICKS(500));
  }
}
```

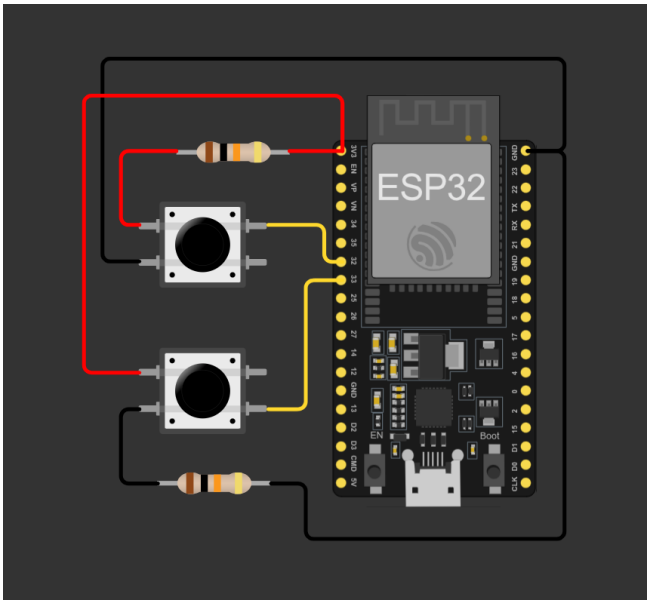
4. klik Build dan Upload.

1.3 Pull-up dan Pull-down: Eksternal vs Internal (ESP32 + Framework Arduino)

1. Ganti isi platformio.ini kembali ke konfigurasi Arduino di bawah. Hapus src/main.c, lalu buat lagi file src/main.cpp

```
[env:esp32dev]
platform = espressif32
board = esp32dev
framework = arduino
```

2. Pasang dua tombol di GPIO 32 dan 33 lengkap dengan resistornya, sesuai Skema di bawah.



3. Tulis Kode di bawah pada src/main.cpp, klik Build dan Upload, lalu coba tekan kedua tombol dan catat hasilnya dari serial monitor.

```
#include <Arduino.h>

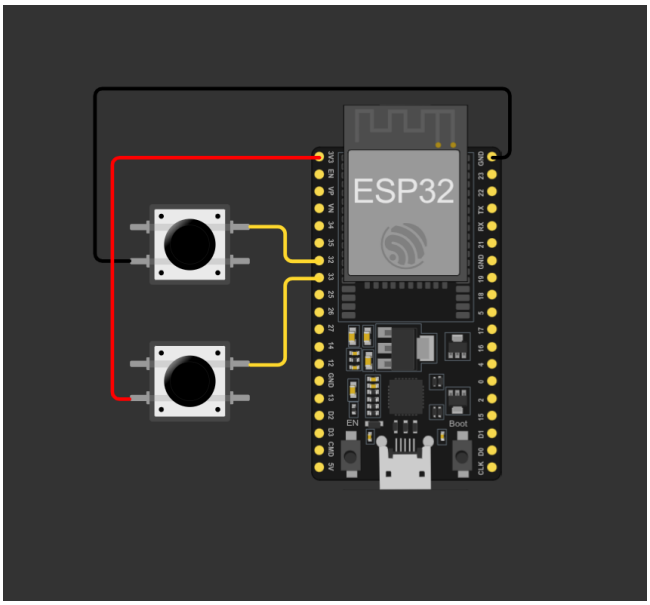
#define BUTTON_PULLUP_PIN 32   // pull-up eksternal
#define BUTTON_PULLDOWN_PIN 33 // pull-down eksternal

void setup() {
  Serial.begin(115200);
  pinMode(BUTTON_PULLUP_PIN, INPUT);   // resistor pull-up dipasang eksternal, bukan INPUT_PULLUP internal
  pinMode(BUTTON_PULLDOWN_PIN, INPUT); // resistor pull-down dipasang eksternal
}

void loop() {
  int stateUp = digitalRead(BUTTON_PULLUP_PIN);
  int stateDown = digitalRead(BUTTON_PULLDOWN_PIN);

  Serial.printf("[EKSTERNAL] Pull-up (GPIO32): %s | Pull-down (GPIO33): %s\n",
                stateUp == LOW ? "DITEKAN" : "TIDAK DITEKAN",
                stateDown == HIGH ? "DITEKAN" : "TIDAK DITEKAN");
  delay(200);
}
```

4. Lepas kedua resistor sesuai skema di bawah, tombol tetap terpasang.



5. Ganti isi src/main.cpp dengan Kode Program B (Internal) di bawah, klik Build dan Upload, lalu coba tombol yang sama dan catat hasilnya dari serial monitor.

```
#include <Arduino.h>

#define BUTTON_PULLUP_PIN 32   // pull-up internal, pin sama seperti Kode Program A
#define BUTTON_PULLDOWN_PIN 33 // pull-down internal, pin sama seperti Kode Program A
```

```
5
6  void setup() {
7    Serial.begin(115200);
8    pinMode(BUTTON_PULLUP_PIN, INPUT_PULLUP);     // pull-up internal ESP32, tanpa
       resistor eksternal
9    pinMode(BUTTON_PULLDOWN_PIN, INPUT_PULLDOWN); // pull-down internal ESP32,
      tanpa resistor eksternal
10 }
11
12 void loop() {
13   int stateUp = digitalRead(BUTTON_PULLUP_PIN);
14   int stateDown = digitalRead(BUTTON_PULLDOWN_PIN);
15
16   Serial.printf("[INTERNAL] Pull-up (GPIO32): %s | Pull-down (GPIO33): %s\n",
17                 stateUp == LOW ? "DITEKAN" : "TIDAK DITEKAN",
18                 stateDown == HIGH ? "DITEKAN" : "TIDAK DITEKAN");
19   delay(200);
20 }
```

1.4 Debouncing pada Input Tombol GPIO (ESP32 + Framework Arduino)

1. Pasang tombol sesuai skema di atas

2. Tulis kode di bawah. Kode ini punya dua penghitung: counterNoDebounce (tanpa penyaring getaran tombol) dan counterWithDebounce (dengan penyaring)

```
1  #include <Arduino.h>
2
3  #define BUTTON_PIN 25
4  #define DEBOUNCE_DELAY 50 // ms
5
6  // Tanpa debouncing
7  int rawLastState = HIGH;
8  unsigned long counterNoDebounce = 0;
9
10 // Dengan debouncing
11 int lastFlickerableState = HIGH;
12 int stableState = HIGH;
13 unsigned long lastDebounceTime = 0;
14 unsigned long counterWithDebounce = 0;
15
16 void setup() {
17   Serial.begin(115200);
18   pinMode(BUTTON_PIN, INPUT_PULLUP);
19 }
20
21 void loop() {
22   int reading = digitalRead(BUTTON_PIN);
23
24   // ===== Tanpa Debouncing =====
25   // Langsung dihitung setiap transisi HIGH->LOW, tanpa validasi waktu
26   if (reading == LOW && rawLastState == HIGH) {
27     counterNoDebounce++;
28   }
```

```
  rawLastState = reading;

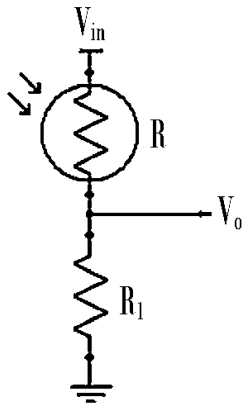
  // ===== Dengan Debouncing =====
  if (reading != lastFlickerableState) {
    lastDebounceTime = millis();
  }
  if ((millis() - lastDebounceTime) > DEBOUNCE_DELAY) {
    if (reading != stableState) {
      stableState = reading;
      if (stableState == LOW) {
        counterWithDebounce++;
      }
    }
  }
  lastFlickerableState = reading;

  // ===== Tampilkan kedua nilai secara bersamaan =====
  static unsigned long lastPrintedNoDebounce = 0, lastPrintedWithDebounce = 0;
  if (counterNoDebounce != lastPrintedNoDebounce || counterWithDebounce !=
    lastPrintedWithDebounce) {
    Serial.printf("Tanpa Debounce: %lu\t| Dengan Debounce: %lu\n",
    counterNoDebounce, counterWithDebounce);
    lastPrintedNoDebounce = counterNoDebounce;
    lastPrintedWithDebounce = counterWithDebounce;
  }
}
```

3. Tekan tombol satu kali, lalu bandingkan nilai kedua penghitung di Serial Monitor
4. Ulangi dengan cara menekan yang berbeda (cepat dan lambat). counterNoDebounce bisa naik lebih dari 1 untuk satu tekanan, sedangkan counterWithDebounce seharusnya naik tepat 1. Jika keduanya terlihat sama, coba tekan lagi karena getaran tombol tidak selalu muncul
5. Ubah DEBOUNCE_DELAY menjadi 10 (10 ms), klik Build dan Upload, lalu tekan tombol lagi
6. Ubah menjadi 200 (200 ms), klik Build dan Upload, lalu tekan tombol dengan cepat berulang kali
7. Bandingkan ketiga nilai (10, 50, dan 200 ms). Analisis untung ruginya: makin besar nilainya makin bersih dari getaran, tetapi tombol makin lambat merespons

1.5 Akses GPIO Analog: Pembacaan ADC dengan LDR (ESP32 + Framework Arduino)

1. Pasang LDR bersama resistor sesuai skema, lalu hubungkan titik tengah keduanya ke GPIO 34



2. Tulis program yang membaca nilai ADC mentah dan mengubahnya menjadi tegangan

```
#include <Arduino.h>

#define LDR_PIN 34

void setup() {
  Serial.begin(115200);
}

void loop() {
  int raw = analogRead(LDR_PIN);
  float voltage = raw * (3.3 / 4095.0);

  Serial.printf("LDR raw: %d | Tegangan: %.2fV\n", raw, voltage);
  delay(300);
}
```

3. Klik Build dan Upload, lalu buka Serial Monitor

4. Tutup LDR dengan tangan, lalu sinari dengan lampu.

5. Analisis mengapa sinyal analog bisa menunjukkan banyak kondisi, tidak hanya HIGH atau LOW

2 Analisis Hasil Percobaan

2.1 Pengenalan STM32 Blackpill (Blink)

Pada percobaan dapat dipelajari bahwa STM32 memerlukan ST Link untuk dapat berkomunikasi melalui USB. Tanpa adanya ST Link, tidak dapat dilakukan komunikasi serial untuk flash program dan memonitor serial output. Pada percobaan ini juga ditemukan sebuah kendala di mana PlatformIO tidak mengkonfigurasi board dengan benar. Pemecahan masalah yang kami lakukan adalah dengan melakukan refresh terhadap VScode sehingga konfigurasi baru terbaca. Ada kemungkinan perubahan file tidak terbaca oleh PlatformIO sehingga perlu diluncurkan ulang untuk membersihkan cache file lama dan platformIO membaca file baru.

2.2 Pengenalan ESP32 dengan Framework ESP-IDF (Blink)

Pada percobaan ini dilakukan aktifasi LED berkedip pada ESP menggunakan framework pengembangan ESP-IDF. Ditemukan masalah berupan kegagalan build dikarenakan terjadi kesalahan format file. Pada framework ESP-IDF digunakan bahasa pemrograman C tidak layaknya framework Arduino yang menggunakan bahasa pemrograman C++. Hal ini menyebabkan kami memerlukan mengganti format platform.ini dan mengganti src/main.cpp menjadi src/main.c.

2.3 Pull-up dan Pull-down: Eksternal vs Internal (ESP32 + Framework Arduino)

Pada percobaan ini dapat terlihat bahwa pada keadaan default (tombol tidak ditekan) hasil dari pull-down adalah LOW dan hasil dari pull-up adalah HIGH. Tidak ada perbedaan antara penggunaan konfigurasi internal maupun eksternal. Penggunaan internal jauh lebih sederhana dikarenakan tidak memerlukan resistor eksternal. Kelebihan dari penggunaan pull-up dan pull-down eksternal yang bisa saya analisa adalah kemungkinan penggunaan sumber tegangan eksternal. Penggunaan sumber tegangan eksternal memungkinkan board tidak terbebani dan kemungkinan overheat lebih kecil.

2.4 Debouncing pada Input Tombol GPIO (ESP32 + Framework Arduino)

Pada masukkan dengan debounce dapat terlihat bahwa hitungannya lebih tinggi daripada tanpa debounce. Hal ini menunjukkan bahwa terjadi pembacaan berulang tanpa adanya debounce. Sementara itu, debounce di 200ms menyebabkan tombol harus ditekan agak lama agar dapat memberi masukkan yang valid. Pada saat 10ms hasil perhitungan debounce lebih besar dibandingkan saat debounce hanya menggunakan 50ms. Hal ini dikarenakan waktu tunggu untuk masukkan valid lebih pendek sehingga ada kemungkinan pantulan yang terbaca oleh 10ms. Dapat dipelajari bahwa debounce terlalu pendek menyebabkan adanya beberapa sinyal pantulan yang masuk, tetapi bila waktu debounce terlalu tinggi, dibutuhkan waktu yang lama agar masukkan terhitung.

2.5 Akses GPIO Analog: Pembacaan ADC dengan LDR (ESP32 + Framework Arduino)

Pada percobaan ini dilakukan pembacaan perubahan tegangan yang dihasilkan oleh perubahan resistansi LDR. LDR menyebabkan perubahan tegangan sesuai perbandingan resistansi pada sirkuit. Perubahan ini bersifat kontinu dan tidak diskrit satu atau nol. Hasil yang dibaca oleh ESP adalah 0.0V (saat ditutup) hingga 0.94V (saat disinari senter HP).

3 Hasil Tugas Modul

Berikut merupakan hasil tugas modul beserta kodenya.

```
1
2 #define ROWS_COUNT 3
3 #define COLS_COUNT 3
4
5 #define TIME_DEBOUNCE 10
6
7 int BUTTON_ROWS[ROWS_COUNT] = {11, 12, 13};
```

```
int BUTTON_COLS[COLS_COUNT] = {2, 3, 4};
int LED_ROWS[ROWS_COUNT]    = {8, 9, 10};
int LED_COLS[COLS_COUNT]    = {5, 6, 7};

void MATRIX_BUTTON_PRESSED(int R, int C){

  for (int i = 0; i < COLS_COUNT; i++) {
    digitalWrite(LED_COLS[i], HIGH);
  }
  for (int i = 0; i < ROWS_COUNT; i++) {
    digitalWrite(LED_ROWS[i], LOW);
  }
  digitalWrite(LED_ROWS[R], HIGH);
  digitalWrite(LED_COLS[C], LOW);

}


void setup(){
  for(int R = 0; R < ROWS_COUNT; R++){
    pinMode(BUTTON_ROWS[R], OUTPUT);
    digitalWrite(BUTTON_ROWS[R], HIGH);
  }
  for(int C = 0; C < COLS_COUNT; C++){
    pinMode(BUTTON_COLS[C], INPUT_PULLUP);
  }

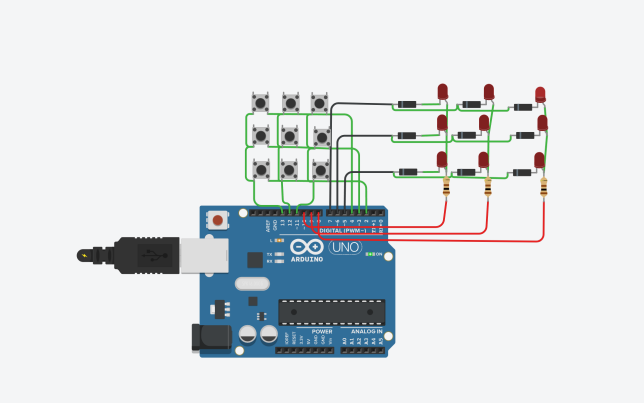
  for(int R = 0; R < ROWS_COUNT; R++){
    pinMode(LED_ROWS[R], OUTPUT);
    digitalWrite(LED_ROWS[R], LOW);
  }
  for(int C = 0; C < COLS_COUNT; C++){
    pinMode(LED_COLS[C], OUTPUT);
    digitalWrite(LED_COLS[C], HIGH);
  }

  Serial.begin(9600);
}


void loop(){
  for(int R = 0; R < ROWS_COUNT; R++){
    digitalWrite(BUTTON_ROWS[R], LOW);

    for(int C = 0; C < COLS_COUNT; C++){
      if(digitalRead(BUTTON_COLS[C]) == LOW){
        delay(TIME_DEBOUNCE);
        if(digitalRead(BUTTON_COLS[C]) == LOW){
          MATRIX_BUTTON_PRESSED(R, C);
        }
      }
    }
    digitalWrite(BUTTON_ROWS[R], HIGH);
    MATRIX_BUTTON_PRESSED(4, 4);
```

```
62    }
63 }
```



1. Debounce tetap diperlukan dikarenakan setiap tombol yang ada merupakan tombol fisik yang dapat menyebabkan getaran. Getaran ini dapat menyebabkan output ke LED tidak stabil dan berpotensi merusak LED. Oleh karena itu debouncing tetap digunakan untuk menghindarinya.

2. Tombol perlu diberikan pull up untuk menghindari kondisi floating. Kondisi floating dapat menyebabkan kesalahan pembacaan. Selain itu, kondisi floating juga dapat menyebabkan korsleting saat tombol ditekan dikarenakan tidak adanya tegangan yang pasti.

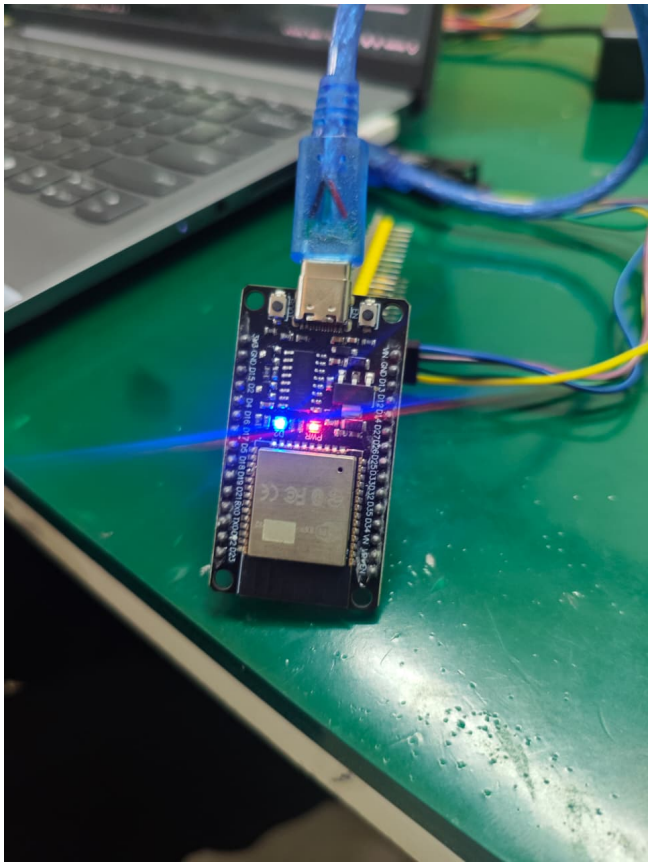
3. Pembacaan kondisi tombol hanya berupa apakah tombol tertekan atau tidak. Sementara itu, pembacaan ADC pada LDR adalah pembacaan perubahan tegangan yang disebabkan oleh perubahan resistansi dari LDR. Resistansi LDR bersifat kontinu sesuai intensitas cahaya. Hal ini menyebabkan data yang dibaca ADC tidak hanya antara high atau low.

4 Kesimpulan

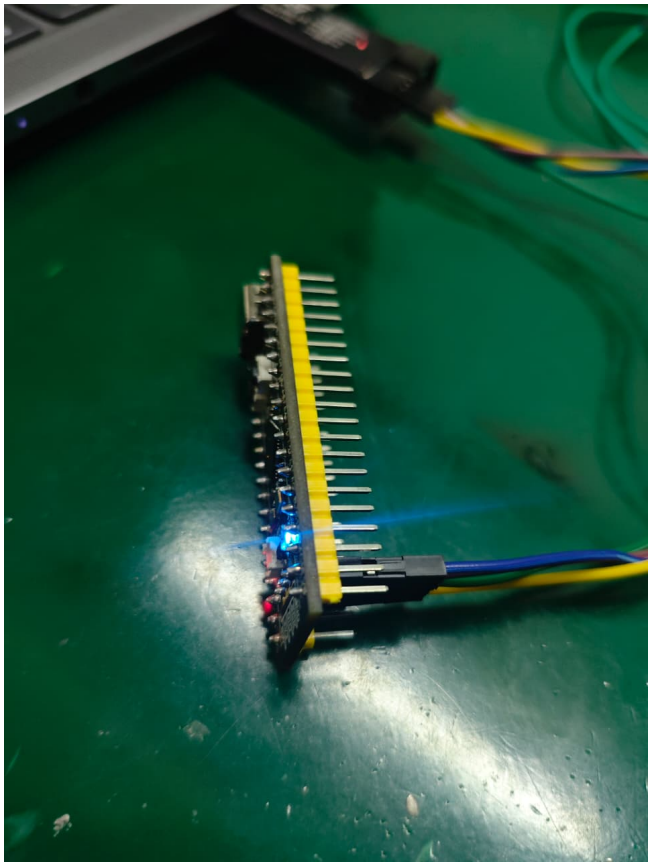
Dari praktikum ini dapat disimpulkan bahwa penggunaan ESP32 dan STM32 dapat menggunakan platformIO dengan framwork berbeda tergantung konfigurasi pada file platform.ini. Selain itu dipelajari pula bahwa kondisi tombol pada mikrokontroller tidak boleh mengalami floating supaya pembacaan input selalu stabil. Selain itu, tombol juga perlu diberikan debounce untuk menghindari pembacaan berulang akibat getaran tombol. Pada praktikum ini juga dipelajari perbedaan antara pembacaan input digital dan analog (ADC).

5 Lampiran

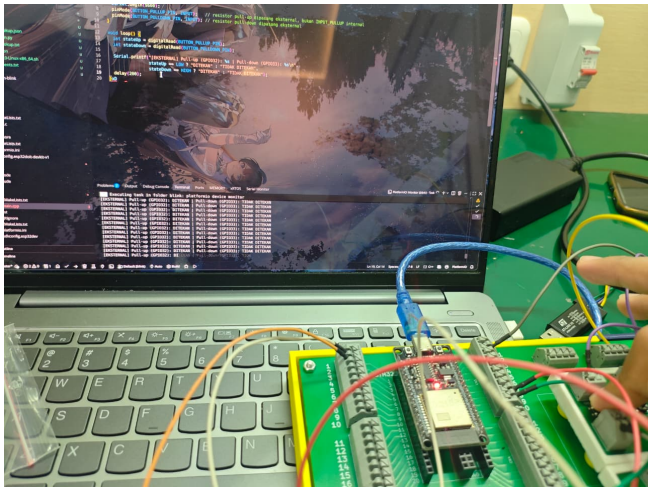
5.1 Dokumentasi saat praktikum



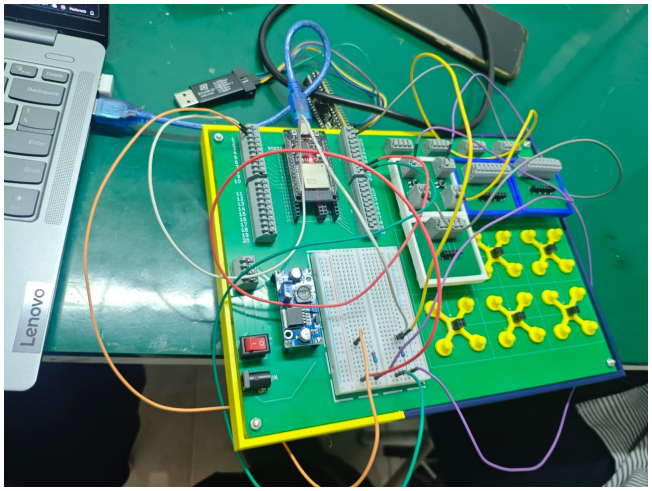
Gambar 1: Percobaan 1



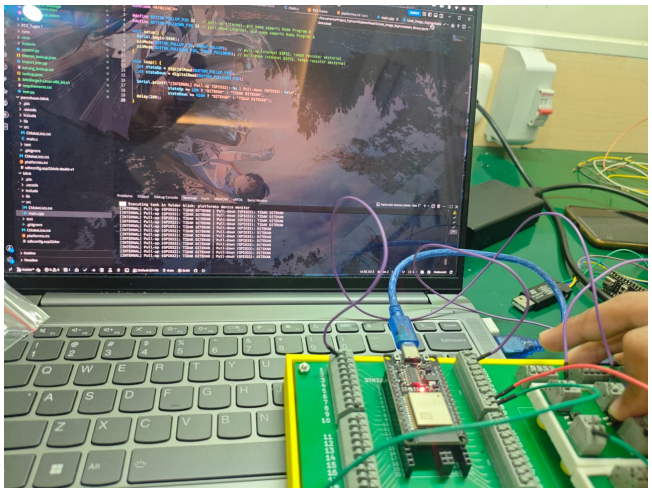
Gambar 2: Percobaan 1



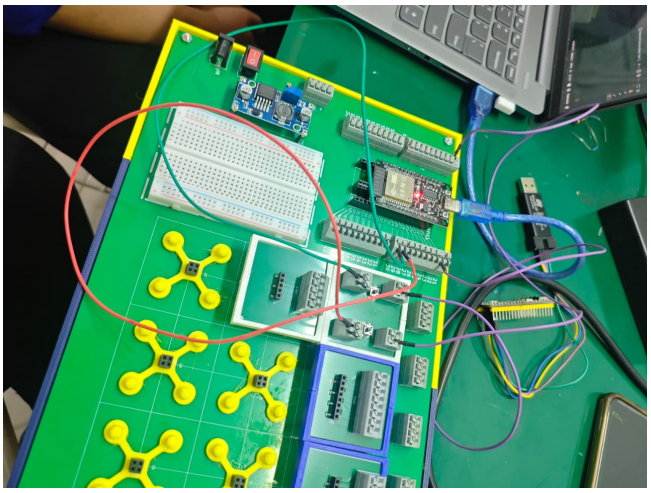
Gambar 3: Pull-up/Pull-down eksternal



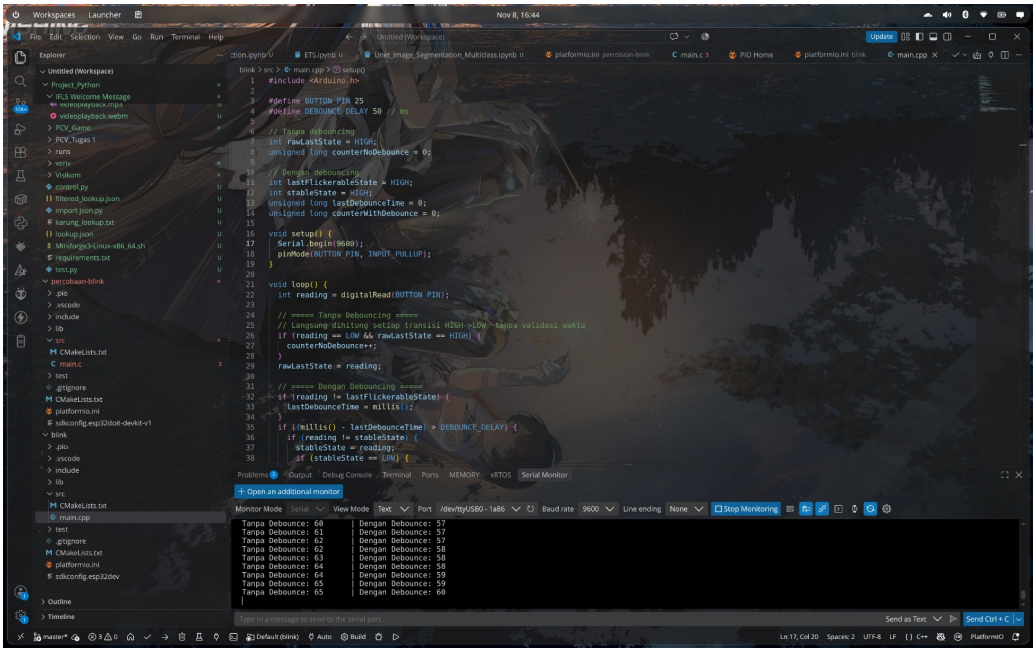
Gambar 4: Skema Pull-up/Pull-down Eksternal



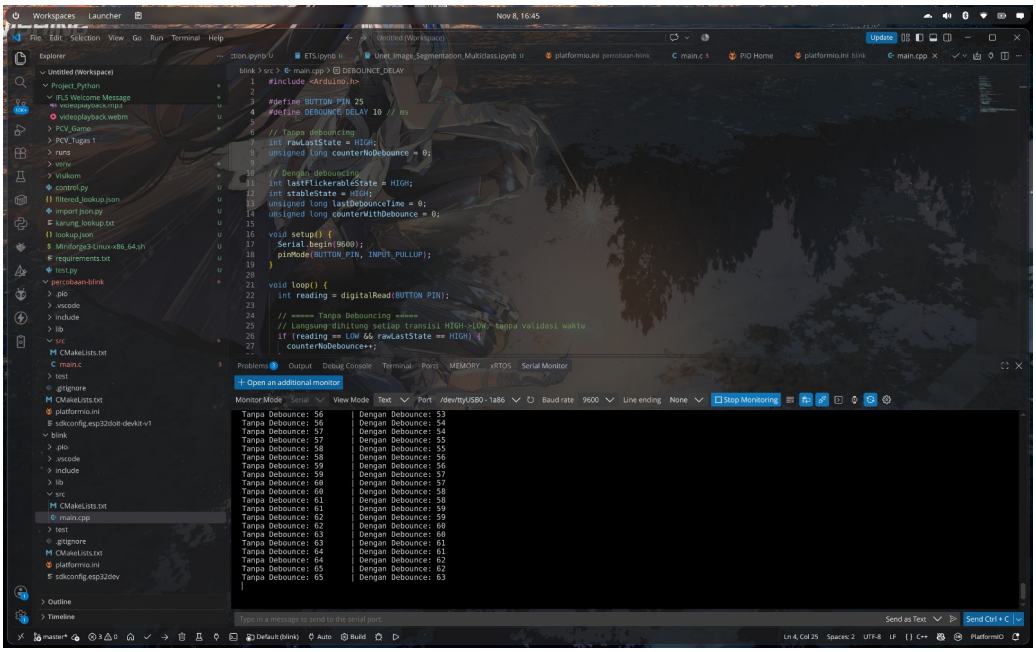
Gambar 5: Pull-up/pull-down internal



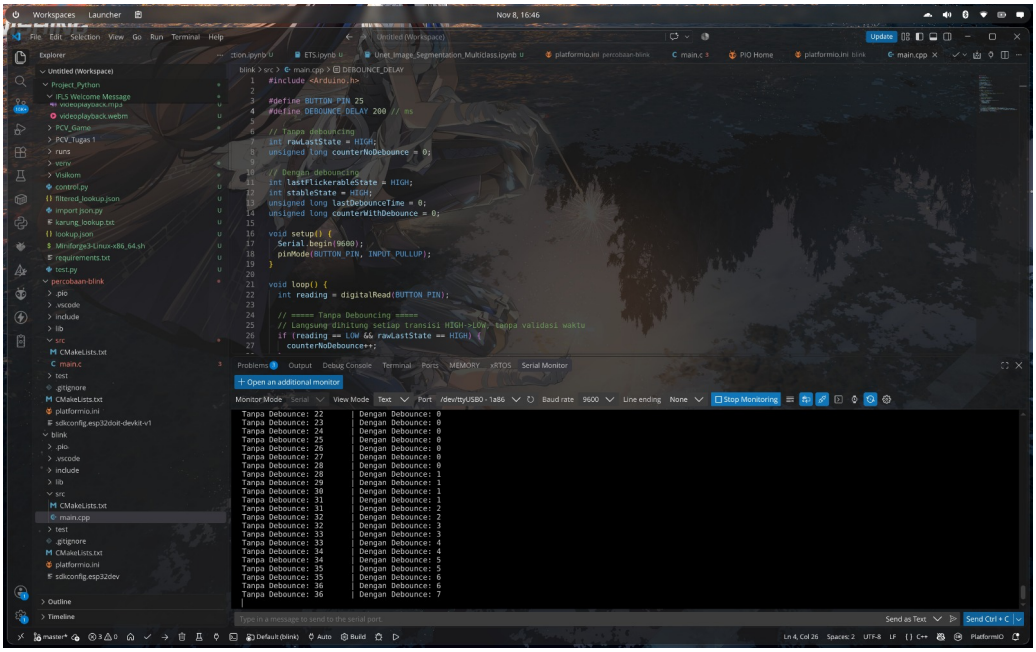
Gambar 6: Skema Pull-up/pull-down internal



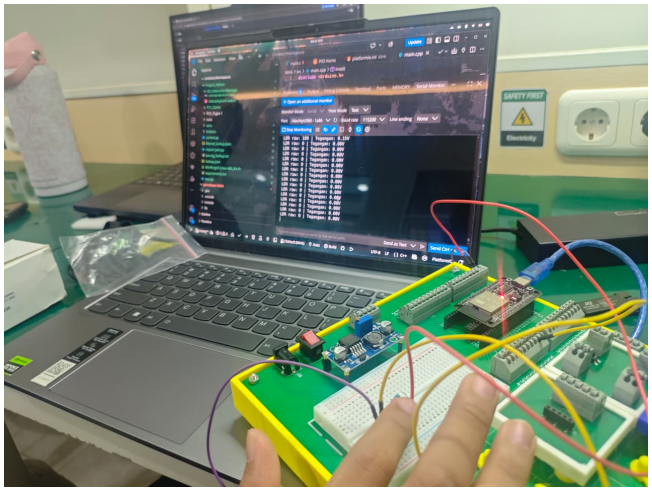
Gambar 7: Debounce 50ms



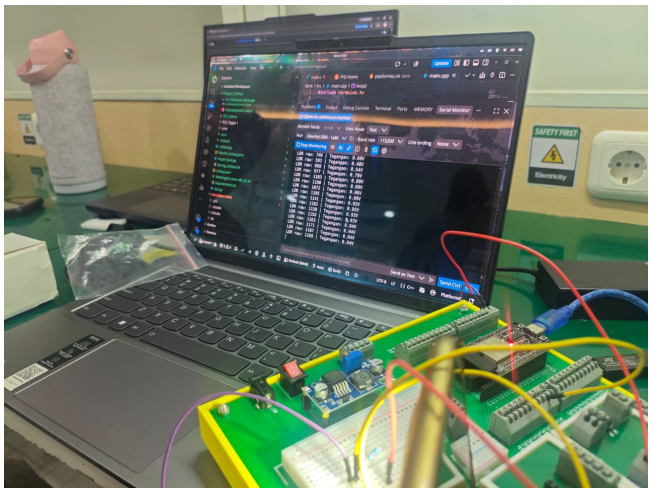
Gambar 8: Debounce 10ms



Gambar 9: Debounce 200ms



Gambar 10: LDR ditutup



Gambar 11: LDR disinari